KARPAGAM

COLLEGE OF ENGINEERING

Rediscover | Refine | Redefine
Accredited by NAAC with 'A+' Grade | Autonomous | Affiliated to Anna University
(An ISO 9001:2015 and ISO 14001:2015 Certified Institution)

ACADEMIC YEAR: 2026-27

ASSIGNMENT FRONT SHEET

Name of the Student:	**MEGA VARSAN VE**
Register Number:	**717825I231**
Course Code & Title:	**23CSR306 JAVA PROGRAMMING**
Year / Dept. / Sem / Section:	**II / AD / III / B**
Assignment:	**II – OOP Mini Project (any one of 35)**
Specification No. & Title:	**Specification 1: Inventory and Billing System**
GitHub Repository Link:	**https://github.com/tcgmegavarsan-sys/Java-Programming-Mini-Project-Assignment-II**
Date of Issue:	**06-10-2026**
Date of Submission:	**25-10-2026**

Marks Details

CO / K Level	Maximum Marks	Marks Obtained
CO4 / K3	50	
CO5 / K3	50	
Total	**100**	

Course In-charge

KARPAGAM
COLLEGE OF ENGINEERING
Rubrics for Assignment Evaluation

Assignment Number	: II	**Date**	: 25-10-2026
Course Code & Title	: 23CSR306 Java Programming	**Specification No. & Title**	: Specification 1: Inventory and Billing System
Register Number	: 717825I231	**Name of the Student**	: MEGA VARSAN VE
Name of the Evaluator	:	**GitHub Link**	: https://github.com/tcgmegavarsan-sys/Java-Programming-Mini-Project-Assignment-II

Marks for each criterion = Level (1 – 5) × Weightage. Total: 100 (CO4: 50 + CO5: 50).

Criterion	CO	Wt.	Exemplary (5)	Excellent (4)	Adequate (3)	Average (2)	Inadequate (1)	Max	Marks
Program Correctness & Business Rules	CO4	4	All menu options work; every business rule enforced correctly.	All options work; minor gaps in one rule.	Main flow works; some rules missing.	Partially working; several rules not handled.	Does not run or most features missing.	**20**	
Input Validation & Exception Handling	CO4	3	All inputs validated; custom exceptions; program never crashes.	Most inputs validated with try-catch.	Basic validation; some invalid inputs not handled.	Little validation; crashes on bad input.	No validation or exception handling.	**15**	
Menu-driven Interface & Expected Outputs	CO4	3	Clear menu; all expected outputs (bills/ receipts/status) shown neatly.	Clear menu; most expected outputs shown.	Menu works; outputs basic or incomplete.	Menu unclear; few outputs shown.	No proper menu or outputs.	**15**	
OOP Design (Encapsulation, Inheritance, Aggregation, Polymorphism)	CO5	4	All four concepts applied meaningfully; all required classes present.	Concepts applied well; minor design gaps.	Classes present; concepts used in a basic way.	Weak design; some required classes missing.	No OOP design; mostly procedural.	**20**	
Code Quality & Package Structure	CO5	2	Clean packages, naming, indentation and comments.	Good structure with few lapses.	Acceptable structure; average readability.	Poor naming / structure.	Single file; unreadable code.	**10**	
GitHub Repository & README	CO5	1	Complete code with clear README (problem, run steps, name, roll no.).	Complete code; README missing minor details.	Code present; README incomplete.	Partial code; no README.	No repository.	**5**	
Viva / Explanation	CO5	3	Explains design, rules and code clearly; answers all questions.	Explains well; answers most questions.	Explains main logic; answers some questions.	Limited explanation of own code.	Unable to explain the code.	**15**	
CO4 Total (Criteria 1 – 3)								**50**	
CO5 Total (Criteria 4 – 7)								**50**	
Total Marks								**100**	

Signature of the Evaluator with date

23CSR306 JAVA PROGRAMMING

Assignment – II Submission Template

Specification 1: Inventory and Billing System

1. Assignment Question

(Complete specification without editing or shortening it)

Specification 1: Inventory and Billing System

Problem Statement:

Design and implement a Java console application for an Inventory and Billing System that handles product purchases, sales, customer types, and a reward points system for premium customers. The application should demonstrate object-oriented principles (encapsulation, inheritance, aggregation, polymorphism) and maintain accurate stock quantities.

Class Requirements:

Product, ProductTaxCategory, Customer, PremiumCustomer, SaleDetail, Sale, PurchaseDetail, Purchase

Business Rules:

1. Stock must be verified before confirming a sale. If insufficient stock is available, the sale should not proceed.
2. Stock quantity must be updated immediately after a sale or purchase.
3. Premium customers earn 1 point for every Rs.100 spent (after discounts, before tax). Once points reach 200 or more, a discount of Rs.10 per 100 points is applied, and redeemed points are deducted.
4. Regular customers do not receive points or discounts.
5. Sale and purchase details should link directly to the associated products.

Menu Options:

Add Product / Add Customer / Make a Purchase / Make a Sale / Display Products / Exit

Expected Output:

- Sale bill with product list, quantities, prices, taxes, discounts (if applicable), total amount and updated reward points (for premium customers).
- Updated stock after every sale or purchase.

OOP Architecture & Business Rule Implementation Mapping:

OOP Concept	Implementation in Inventory and Billing System
Encapsulation	Private fields in Product, Customer, Sale, Purchase, etc. with verified public getters/setters preventing invalid states.
Inheritance	PremiumCustomer extends Customer. Inherits baseline fields and adds rewardPoints and membershipTier.
Polymorphism	calculateDiscount(), calculatePointsEarned(), and redeemPointsForDiscount() overridden in PremiumCustomer for dynamic dispatch.
Aggregation	SaleDetail and PurchaseDetail directly reference independent Product instances (Business Rule 5). Sale aggregates Customer.
Composition	Sale manages List<SaleDetail>; Purchase manages List<PurchaseDetail>.
Exception Handling	Custom checked exceptions (InsufficientStockException, ProductNotFoundException, etc.) prevent crashes on invalid inputs.

2. Java Source Code

Paste the complete code from Eclipse, package by package – model, service, util, exception, main. Consolas / Courier New.

File: model/ProductTaxCategory.java

```java
package com.inventory.model;

import java.io.Serializable;

/**
 * Represents the tax category applicable to products in the inventory system.
 * Encapsulates GST/tax rates and category metadata.
 */
public class ProductTaxCategory implements Serializable {
    private static final long serialVersionUID = 1L;

    private String categoryId;
    private String categoryName;
    private double gstRate; // in percentage, e.g., 5.0, 12.0, 18.0

    // Standard pre-defined tax categories
    public static final ProductTaxCategory EXEMPT = new ProductTaxCategory("TC01", "Exempt (0%)", 0.0);
    public static final ProductTaxCategory ESSENTIAL = new ProductTaxCategory("TC02", "Essential (5%)", 5.0);
    public static final ProductTaxCategory STANDARD = new ProductTaxCategory("TC03", "Standard (12%)", 12.0);
    public static final ProductTaxCategory ELECTRONICS = new ProductTaxCategory("TC04", "Electronics (18%)", 18.0);
    public static final ProductTaxCategory LUXURY = new ProductTaxCategory("TC05", "Luxury (28%)", 28.0);

    public ProductTaxCategory() {
    }

    public ProductTaxCategory(String categoryId, String categoryName, double gstRate) {
        this.categoryId = categoryId;
        this.categoryName = categoryName;
        this.gstRate = Math.max(0.0, gstRate);
    }

    public String getCategoryId() {
        return categoryId;
    }

    public void setCategoryId(String categoryId) {
        this.categoryId = categoryId;
    }

    public String getCategoryName() {
        return categoryName;
    }

    public void setCategoryName(String categoryName) {
        this.categoryName = categoryName;
    }

    public double getGstRate() {
        return gstRate;
    }

    public void setGstRate(double gstRate) {
        this.gstRate = Math.max(0.0, gstRate);
    }

    @Override
    public String toString() {
        return categoryName + " [" + gstRate + "%]";
    }
}

```

File: model/Product.java

```java
package com.inventory.model;

import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Represents an inventory product.
 * Implements encapsulation and links directly to a ProductTaxCategory.
 */
public class Product implements Serializable {
    private static final long serialVersionUID = 1L;

    private String productId;
```

(continued) File: model/Product.java

```
    private String name;
    private double price;
    private int stockQuantity;
    private ProductTaxCategory taxCategory;

    public Product() {
    }

    public Product(String productId, String name, double price, int stockQuantity, ProductTaxCategory taxCategory)
            throws InvalidInputException {
        if (productId == null || productId.trim().isEmpty()) {
            throw new InvalidInputException("Product ID cannot be null or empty.");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Product Name cannot be null or empty.");
        }
        if (price < 0) {
            throw new InvalidInputException("Product price cannot be negative.");
        }
        if (stockQuantity < 0) {
            throw new InvalidInputException("Stock quantity cannot be negative.");
        }
        this.productId = productId.trim().toUpperCase();
        this.name = name.trim();
        this.price = price;
        this.stockQuantity = stockQuantity;
        this.taxCategory = (taxCategory != null) ? taxCategory : ProductTaxCategory.STANDARD;
    }

    public String getProductId() {
        return productId;
    }

    public void setProductId(String productId) throws InvalidInputException {
        if (productId == null || productId.trim().isEmpty()) {
            throw new InvalidInputException("Product ID cannot be empty.");
        }
        this.productId = productId.trim().toUpperCase();
    }

    public String getName() {
        return name;
    }

    public void setName(String name) throws InvalidInputException {
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Product name cannot be empty.");
        }
        this.name = name.trim();
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) throws InvalidInputException {
        if (price < 0) {
            throw new InvalidInputException("Price cannot be negative.");
        }
        this.price = price;
    }

    public int getStockQuantity() {
        return stockQuantity;
    }

    public void setStockQuantity(int stockQuantity) throws InvalidInputException {
        if (stockQuantity < 0) {
            throw new InvalidInputException("Stock quantity cannot be negative.");
        }
        this.stockQuantity = stockQuantity;
    }

    public ProductTaxCategory getTaxCategory() {
        return taxCategory;
    }

    public void setTaxCategory(ProductTaxCategory taxCategory) {
        this.taxCategory = (taxCategory != null) ? taxCategory : ProductTaxCategory.STANDARD;
    }

    /**
```

(continued) File: model/Product.java

```
     * Increases stock quantity immediately after a purchase.
     * Enforces Business Rule 2.
     */
    public synchronized void addStock(int quantity) throws InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Quantity to add must be positive.");
        }
        this.stockQuantity += quantity;
    }

    /**
     * Verifies and decreases stock quantity immediately after a sale.
     * Enforces Business Rules 1 & 2.
     */
    public synchronized void deductStock(int quantity) throws InsufficientStockException, InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Quantity to deduct must be positive.");
        }
        if (this.stockQuantity < quantity) {
            throw new InsufficientStockException(this.productId, quantity, this.stockQuantity);
        }
        this.stockQuantity -= quantity;
    }

    /**
     * Calculates tax amount per unit of this product based on its tax category.
     */
    public double getTaxAmountPerUnit() {
        if (taxCategory == null) return 0.0;
        return (price * taxCategory.getGstRate()) / 100.0;
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-22s | Rs. %-9.2f | Stock: %-5d | Tax: %s",
                productId, name, price, stockQuantity, (taxCategory != null ? taxCategory.toString() : "None"));
    }
}

```

File: model/Customer.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Base class representing a customer in the inventory and billing system.
 * Demonstrates Encapsulation and acts as the parent for polymorphic customer types.
 */
public class Customer implements Serializable {
    private static final long serialVersionUID = 1L;

    private String customerId;
    private String name;
    private String contactNumber;
    private String email;

    public Customer() {
    }

    public Customer(String customerId, String name, String contactNumber, String email)
            throws InvalidInputException {
        if (customerId == null || customerId.trim().isEmpty()) {
            throw new InvalidInputException("Customer ID cannot be empty.");
        }
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Customer Name cannot be empty.");
        }
        this.customerId = customerId.trim().toUpperCase();
        this.name = name.trim();
        this.contactNumber = (contactNumber != null) ? contactNumber.trim() : "";
        this.email = (email != null) ? email.trim() : "";
    }

    public String getCustomerId() {
        return customerId;
    }

    public void setCustomerId(String customerId) throws InvalidInputException {
```

(continued) File: model/Customer.java

```
        if (customerId == null || customerId.trim().isEmpty()) {
            throw new InvalidInputException("Customer ID cannot be empty.");
        }
        this.customerId = customerId.trim().toUpperCase();
    }

    public String getName() {
        return name;
    }

    public void setName(String name) throws InvalidInputException {
        if (name == null || name.trim().isEmpty()) {
            throw new InvalidInputException("Customer Name cannot be empty.");
        }
        this.name = name.trim();
    }

    public String getContactNumber() {
        return contactNumber;
    }

    public void setContactNumber(String contactNumber) {
        this.contactNumber = (contactNumber != null) ? contactNumber.trim() : "";
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = (email != null) ? email.trim() : "";
    }

    /**
     * Polymorphic method returning the customer category.
     */
    public String getCustomerType() {
        return "Regular";
    }

    /**
     * Polymorphic method calculating eligible discount before tax.
     * Business Rule 4: Regular customers do not receive points or discounts.
     */
    public double calculateDiscount(double spentBeforeTax) {
        return 0.0;
    }

    /**
     * Polymorphic method calculating earned reward points.
     * Business Rule 4: Regular customers do not receive points or discounts.
     */
    public int calculatePointsEarned(double amountAfterDiscountBeforeTax) {
        return 0;
    }

    /**
     * Deducts redeemed reward points from the customer balance.
     * No-op for regular customers.
     */
    public int redeemPointsForDiscount(double discountApplied) {
        return 0;
    }

    /**
     * Returns current reward points. Regular customers have 0.
     */
    public int getRewardPoints() {
        return 0;
    }

    /**
     * Adds reward points to customer account. No-op for regular customers.
     */
    public void addRewardPoints(int points) {
        // Regular customers do not collect points
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %-12s | %-10s | %s",
                customerId, name, contactNumber, getCustomerType(), email);
```

(continued) File: model/Customer.java

```
    }
}

```

File: model/PremiumCustomer.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;

/**
 * Represents a premium customer in the inventory system.
 * Extends Customer to demonstrate Inheritance and Polymorphism.
 * Enforces Business Rule 3:
 * - Earns 1 point for every Rs. 100 spent (after discounts, before tax).
 * - When points reach 200 or more, Rs. 10 discount per 100 points is applied, and points are deducted.
 */
public class PremiumCustomer extends Customer {
    private static final long serialVersionUID = 1L;

    private int rewardPoints;
    private String membershipTier;

    public PremiumCustomer() {
        super();
        this.rewardPoints = 0;
        this.membershipTier = "Gold";
    }

    public PremiumCustomer(String customerId, String name, String contactNumber, String email, int initialPoints)
            throws InvalidInputException {
        super(customerId, name, contactNumber, email);
        if (initialPoints < 0) {
            throw new InvalidInputException("Initial reward points cannot be negative.");
        }
        this.rewardPoints = initialPoints;
        this.membershipTier = determineTier(initialPoints);
    }

    private String determineTier(int points) {
        if (points >= 1000) return "Platinum";
        if (points >= 500) return "Gold";
        return "Silver";
    }

    @Override
    public String getCustomerType() {
        return "Premium (" + membershipTier + ")";
    }

    @Override
    public int getRewardPoints() {
        return rewardPoints;
    }

    public void setRewardPoints(int rewardPoints) throws InvalidInputException {
        if (rewardPoints < 0) {
            throw new InvalidInputException("Reward points cannot be negative.");
        }
        this.rewardPoints = rewardPoints;
        this.membershipTier = determineTier(rewardPoints);
    }

    public String getMembershipTier() {
        return membershipTier;
    }

    public void setMembershipTier(String membershipTier) {
        this.membershipTier = membershipTier;
    }

    /**
     * Polymorphic implementation of discount calculation.
     * Rule 3: Once points reach 200 or more, a discount of Rs. 10 per 100 points is applied.
     */
    @Override
    public double calculateDiscount(double spentBeforeTax) {
        if (this.rewardPoints < 200 || spentBeforeTax <= 0) {
            return 0.0;
        }
        int eligibleBlocks = this.rewardPoints / 100;
```

(continued) File: model/PremiumCustomer.java

```
        double potentialDiscount = eligibleBlocks * 10.0;

        // Discount cannot exceed the gross amount before tax
        if (potentialDiscount > spentBeforeTax) {
            int applicableBlocks = (int) (spentBeforeTax / 10.0);
            return applicableBlocks * 10.0;
        }
        return potentialDiscount;
    }

    /**
     * Polymorphic calculation of reward points earned.
     * Rule 3: Earns 1 point for every Rs. 100 spent (after discounts, before tax).
     */
    @Override
    public int calculatePointsEarned(double amountAfterDiscountBeforeTax) {
        if (amountAfterDiscountBeforeTax < 100.0) {
            return 0;
        }
        return (int) (amountAfterDiscountBeforeTax / 100.0);
    }

    /**
     * Deducts points corresponding to the applied discount.
     * Rule 3: Redeemed points are deducted (100 points per Rs. 10 discount).
     */
    @Override
    public int redeemPointsForDiscount(double discountApplied) {
        if (discountApplied <= 0) {
            return 0;
        }
        int pointsToDeduct = ((int) Math.round(discountApplied / 10.0)) * 100;
        if (pointsToDeduct > this.rewardPoints) {
            pointsToDeduct = this.rewardPoints;
        }
        this.rewardPoints -= pointsToDeduct;
        this.membershipTier = determineTier(this.rewardPoints);
        return pointsToDeduct;
    }

    /**
     * Updates customer reward points with newly earned points.
     */
    @Override
    public void addRewardPoints(int points) {
        if (points > 0) {
            this.rewardPoints += points;
            this.membershipTier = determineTier(this.rewardPoints);
        }
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %-12s | %-10s | %-18s | Points: %d",
                getCustomerId(), getName(), getContactNumber(), "Premium", getEmail(), rewardPoints);
    }
}

```

File: model/PurchaseDetail.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Represents an individual line item in a restock/purchase order.
 * Demonstrates Aggregation by directly linking to an associated Product (Business Rule 5).
 */
public class PurchaseDetail implements Serializable {
    private static final long serialVersionUID = 1L;

    private Product product;
    private int quantity;
    private double unitCostPrice;

    public PurchaseDetail() {
    }

    public PurchaseDetail(Product product, int quantity, double unitCostPrice) throws InvalidInputException {
```

(continued) File: model/PurchaseDetail.java

```
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        if (quantity <= 0) {
            throw new InvalidInputException("Purchase quantity must be greater than zero.");
        }
        if (unitCostPrice < 0) {
            throw new InvalidInputException("Unit cost price cannot be negative.");
        }
        this.product = product;
        this.quantity = quantity;
        this.unitCostPrice = unitCostPrice;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        this.product = product;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) throws InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Purchase quantity must be greater than zero.");
        }
        this.quantity = quantity;
    }

    public double getUnitCostPrice() {
        return unitCostPrice;
    }

    public void setUnitCostPrice(double unitCostPrice) throws InvalidInputException {
        if (unitCostPrice < 0) {
            throw new InvalidInputException("Unit cost price cannot be negative.");
        }
        this.unitCostPrice = unitCostPrice;
    }

    public double getSubtotal() {
        return quantity * unitCostPrice;
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %5d units @ Rs. %-8.2f = Rs. %-10.2f",
                product.getProductId(), product.getName(), quantity, unitCostPrice, getSubtotal());
    }
}
```

File: model/Purchase.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Represents a purchase/restocking transaction from a supplier.
 * Demonstrates Composition by owning a list of PurchaseDetail items.
 * Enforces Business Rule 2: Stock quantity must be updated immediately after a purchase.
 */
public class Purchase implements Serializable {
    private static final long serialVersionUID = 1L;

    private String purchaseId;
    private LocalDateTime purchaseDate;
```

(continued) File: model/Purchase.java

```
    private String supplierName;
    private List<PurchaseDetail> purchaseDetails;
    private double totalAmount;
    private boolean processed;

    public Purchase() {
        this.purchaseDetails = new ArrayList<>();
        this.purchaseDate = LocalDateTime.now();
        this.processed = false;
    }

    public Purchase(String purchaseId, String supplierName) throws InvalidInputException {
        if (purchaseId == null || purchaseId.trim().isEmpty()) {
            throw new InvalidInputException("Purchase ID cannot be empty.");
        }
        if (supplierName == null || supplierName.trim().isEmpty()) {
            throw new InvalidInputException("Supplier name cannot be empty.");
        }
        this.purchaseId = purchaseId.trim().toUpperCase();
        this.supplierName = supplierName.trim();
        this.purchaseDate = LocalDateTime.now();
        this.purchaseDetails = new ArrayList<>();
        this.totalAmount = 0.0;
        this.processed = false;
    }

    public String getPurchaseId() {
        return purchaseId;
    }

    public void setPurchaseId(String purchaseId) throws InvalidInputException {
        if (purchaseId == null || purchaseId.trim().isEmpty()) {
            throw new InvalidInputException("Purchase ID cannot be empty.");
        }
        this.purchaseId = purchaseId.trim().toUpperCase();
    }

    public LocalDateTime getPurchaseDate() {
        return purchaseDate;
    }

    public void setPurchaseDate(LocalDateTime purchaseDate) {
        this.purchaseDate = purchaseDate;
    }

    public String getSupplierName() {
        return supplierName;
    }

    public void setSupplierName(String supplierName) throws InvalidInputException {
        if (supplierName == null || supplierName.trim().isEmpty()) {
            throw new InvalidInputException("Supplier name cannot be empty.");
        }
        this.supplierName = supplierName.trim();
    }

    public List<PurchaseDetail> getPurchaseDetails() {
        return Collections.unmodifiableList(purchaseDetails);
    }

    public double getTotalAmount() {
        return totalAmount;
    }

    public boolean isProcessed() {
        return processed;
    }

    /**
     * Adds an item detail to the purchase.
     */
    public void addPurchaseDetail(PurchaseDetail detail) throws InvalidInputException {
        if (detail == null) {
            throw new InvalidInputException("Purchase detail cannot be null.");
        }
        if (processed) {
            throw new InvalidInputException("Cannot add details to an already processed purchase.");
        }
        purchaseDetails.add(detail);
        totalAmount += detail.getSubtotal();
    }

```

(continued) File: model/Purchase.java

```
    /**
     * Confirms the purchase and immediately updates product stock.
     * Enforces Business Rule 2.
     */
    public synchronized void processPurchase() throws InvalidInputException {
        if (processed) {
            throw new InvalidInputException("Purchase has already been processed.");
        }
        if (purchaseDetails.isEmpty()) {
            throw new InvalidInputException("Cannot process an empty purchase order.");
        }
        for (PurchaseDetail detail : purchaseDetails) {
            detail.getProduct().addStock(detail.getQuantity());
        }
        this.processed = true;
    }

    /**
     * Generates a detailed receipt for the purchase order.
     */
    public String getPurchaseReceipt() {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm:ss");
        StringBuilder sb = new StringBuilder();
        sb.append("================================================================================\n");
        sb.append("                      PURCHASE / RESTOCK RECEIPT                                \n");
        sb.append("================================================================================\n");
        sb.append(String.format("Purchase ID : %-25s Date: %s\n", purchaseId, purchaseDate.format(dtf)));
        sb.append(String.format("Supplier    : %s\n", supplierName));
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append(String.format("%-10s %-24s %8s %12s %14s\n", "Prod ID", "Product Name", "Qty", "Cost (Rs. )", "Subtotal
(Rs. )"));
        sb.append("--------------------------------------------------------------------------------\n");
        for (PurchaseDetail d : purchaseDetails) {
            sb.append(String.format("%-10s %-24s %8d %12.2f %14.2f\n",
                    d.getProduct().getProductId(),
                    d.getProduct().getName(),
                    d.getQuantity(),
                    d.getUnitCostPrice(),
                    d.getSubtotal()));
        }
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append(String.format("TOTAL PURCHASE AMOUNT:                                Rs. %14.2f\n", totalAmount));
        sb.append("--------------------------------------------------------------------------------\n");
        sb.append("UPDATED PRODUCT STOCK LEVELS:\n");
        for (PurchaseDetail d : purchaseDetails) {
            sb.append(String.format("  • %s (%s): Current Available Stock = %d units\n",
                    d.getProduct().getName(),
                    d.getProduct().getProductId(),
                    d.getProduct().getStockQuantity()));
        }
        sb.append("================================================================================\n");
        return sb.toString();
    }
}

```

File: model/SaleDetail.java

```
package com.inventory.model;

import com.inventory.exception.InvalidInputException;
import java.io.Serializable;

/**
 * Represents an individual line item in a sales invoice.
 * Demonstrates Aggregation by directly linking to an associated Product (Business Rule 5).
 */
public class SaleDetail implements Serializable {
    private static final long serialVersionUID = 1L;

    private Product product;
    private int quantity;
    private double unitPrice;
    private double taxRate; // in %

    public SaleDetail() {
    }

    public SaleDetail(Product product, int quantity) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
```

(continued) File: model/SaleDetail.java

```
        if (quantity <= 0) {
            throw new InvalidInputException("Sale quantity must be greater than zero.");
        }
        this.product = product;
        this.quantity = quantity;
        this.unitPrice = product.getPrice();
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public SaleDetail(Product product, int quantity, double unitPrice) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        if (quantity <= 0) {
            throw new InvalidInputException("Sale quantity must be greater than zero.");
        }
        if (unitPrice < 0) {
            throw new InvalidInputException("Unit price cannot be negative.");
        }
        this.product = product;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public Product getProduct() {
        return product;
    }

    public void setProduct(Product product) throws InvalidInputException {
        if (product == null) {
            throw new InvalidInputException("Associated product cannot be null.");
        }
        this.product = product;
        this.taxRate = (product.getTaxCategory() != null) ? product.getTaxCategory().getGstRate() : 0.0;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) throws InvalidInputException {
        if (quantity <= 0) {
            throw new InvalidInputException("Quantity must be greater than zero.");
        }
        this.quantity = quantity;
    }

    public double getUnitPrice() {
        return unitPrice;
    }

    public void setUnitPrice(double unitPrice) throws InvalidInputException {
        if (unitPrice < 0) {
            throw new InvalidInputException("Unit price cannot be negative.");
        }
        this.unitPrice = unitPrice;
    }

    public double getTaxRate() {
        return taxRate;
    }

    /**
     * Calculates gross amount before tax and discount for this line item.
     */
    public double getGrossAmount() {
        return quantity * unitPrice;
    }

    /**
     * Calculates tax amount for this line item based on its tax rate.
     */
    public double getTaxAmount() {
        return (getGrossAmount() * taxRate) / 100.0;
    }

    /**
     * Returns gross amount plus tax for this line item.
     */
    public double getLineTotal() {
        return getGrossAmount() + getTaxAmount();
```

(continued) File: model/SaleDetail.java

```
    }

    @Override
    public String toString() {
        return String.format("%-8s | %-20s | %5d | Rs. %-8.2f | %4.1f%% | Rs. %-8.2f | Rs. %-9.2f",
                product.getProductId(), product.getName(), quantity, unitPrice, taxRate, getTaxAmount(), getLineTotal());
    }
}

```

File: model/Sale.java

```
package com.inventory.model;

import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Represents a completed sales transaction and tax invoice.
 * Demonstrates:
 * - Aggregation: Customer reference (polymorphic dispatch for discounts and reward points)
 * - Composition: List of SaleDetail objects
 * - Business Rules:
 *   1. Atomic stock verification prior to confirmation.
 *   2. Immediate stock decrement upon confirmation.
 *   3. Reward points redemption and accrual for PremiumCustomer.
 *   4. Zero points and zero discount for regular Customer.
 *   5. Sale bill formatting displaying all required breakdown details.
 */
public class Sale implements Serializable {
    private static final long serialVersionUID = 1L;

    private String saleId;
    private LocalDateTime saleDate;
    private Customer customer;
    private List<SaleDetail> saleDetails;

    // Financial totals
    private double grossTotal;
    private double discountAmount;
    private int pointsRedeemed;
    private double subtotalAfterDiscount;
    private double totalTaxAmount;
    private double netTotal;

    // Loyalty points tracking
    private int pointsBeforeSale;
    private int pointsEarned;
    private int pointsAfterSale;

    private boolean processed;

    public Sale() {
        this.saleDetails = new ArrayList<>();
        this.saleDate = LocalDateTime.now();
        this.processed = false;
    }

    public Sale(String saleId, Customer customer) throws InvalidInputException {
        if (saleId == null || saleId.trim().isEmpty()) {
            throw new InvalidInputException("Sale ID cannot be empty.");
        }
        if (customer == null) {
            throw new InvalidInputException("Customer cannot be null for a sale.");
        }
        this.saleId = saleId.trim().toUpperCase();
        this.customer = customer;
        this.saleDate = LocalDateTime.now();
        this.saleDetails = new ArrayList<>();
        this.processed = false;
    }

    public String getSaleId() {
        return saleId;
    }
```

(continued) File: model/Sale.java

```
70
71      public void setSaleId(String saleId) throws InvalidInputException {
72          if (saleId == null || saleId.trim().isEmpty()) {
73              throw new InvalidInputException("Sale ID cannot be empty.");
74          }
75          this.saleId = saleId.trim().toUpperCase();
76      }
77
78      public LocalDateTime getSaleDate() {
79          return saleDate;
80      }
81
82      public void setSaleDate(LocalDateTime saleDate) {
83          this.saleDate = saleDate;
84      }
85
86      public Customer getCustomer() {
87          return customer;
88      }
89
90      public void setCustomer(Customer customer) throws InvalidInputException {
91          if (customer == null) {
92              throw new InvalidInputException("Customer cannot be null.");
93          }
94          this.customer = customer;
95      }
96
97      public List<SaleDetail> getSaleDetails() {
98          return Collections.unmodifiableList(saleDetails);
99      }
100
101     public double getGrossTotal() {
102         return grossTotal;
103     }
104
105     public double getDiscountAmount() {
106         return discountAmount;
107     }
108
109     public int getPointsRedeemed() {
110         return pointsRedeemed;
111     }
112
113     public double getSubtotalAfterDiscount() {
114         return subtotalAfterDiscount;
115     }
116
117     public double getTotalTaxAmount() {
118         return totalTaxAmount;
119     }
120
121     public double getNetTotal() {
122         return netTotal;
123     }
124
125     public int getPointsBeforeSale() {
126         return pointsBeforeSale;
127     }
128
129     public int getPointsEarned() {
130         return pointsEarned;
131     }
132
133     public int getPointsAfterSale() {
134         return pointsAfterSale;
135     }
136
137     public boolean isProcessed() {
138         return processed;
139     }
140
141     /**
142      * Adds an item to the sale before processing.
143      */
144     public void addSaleDetail(SaleDetail detail) throws InvalidInputException {
145         if (detail == null) {
146             throw new InvalidInputException("Sale detail cannot be null.");
147         }
148         if (processed) {
149             throw new InvalidInputException("Cannot add items to a finalized sale.");
150         }
151         saleDetails.add(detail);
```

(continued) File: model/Sale.java

```
    }

    /**
     * Processes and finalizes the sale adhering to all business rules:
     * 1. Verifies stock availability for ALL items before deducting any stock (Rule 1).
     * 2. Calculates gross total across all sale details.
     * 3. Calculates customer discount and points redemption polymorphically (Rules 3 & 4).
     * 4. Deducts stock immediately upon confirmation (Rule 2).
     * 5. Accrues new reward points for premium customer (Rule 3).
     */
    public synchronized void processSale() throws InsufficientStockException, InvalidInputException {
        if (processed) {
            throw new InvalidInputException("This sale has already been processed and finalized.");
        }
        if (saleDetails.isEmpty()) {
            throw new InvalidInputException("Cannot process a sale with no items.");
        }

        // --- BUSINESS RULE 1: Stock verification before confirming sale ---
        for (SaleDetail detail : saleDetails) {
            Product prod = detail.getProduct();
            if (prod.getStockQuantity() < detail.getQuantity()) {
                throw new InsufficientStockException(prod.getProductId(), detail.getQuantity(), prod.getStockQuantity());
            }
        }

        // Calculate gross total (before discount and tax)
        this.grossTotal = 0.0;
        for (SaleDetail detail : saleDetails) {
            this.grossTotal += detail.getGrossAmount();
        }

        // Record initial customer reward points
        this.pointsBeforeSale = customer.getRewardPoints();

        // --- BUSINESS RULE 3 & 4: Polymorphic discount and points handling ---
        this.discountAmount = customer.calculateDiscount(this.grossTotal);
        this.pointsRedeemed = customer.redeemPointsForDiscount(this.discountAmount);

        // Subtotal after discount, before tax
        this.subtotalAfterDiscount = Math.max(0.0, this.grossTotal - this.discountAmount);

        // Calculate tax based on proportion after discount
        double discountRatio = (this.grossTotal > 0.0) ? (this.subtotalAfterDiscount / this.grossTotal) : 1.0;
        this.totalTaxAmount = 0.0;
        for (SaleDetail detail : saleDetails) {
            double lineGross = detail.getGrossAmount();
            double lineTaxable = lineGross * discountRatio;
            double lineTax = (lineTaxable * detail.getTaxRate()) / 100.0;
            this.totalTaxAmount += lineTax;
        }

        // Net total payable
        this.netTotal = this.subtotalAfterDiscount + this.totalTaxAmount;

        // --- BUSINESS RULE 2: Immediate stock deduction ---
        for (SaleDetail detail : saleDetails) {
            detail.getProduct().deductStock(detail.getQuantity());
        }

        // --- BUSINESS RULE 3: Reward points earning for Premium Customers ---
        // Earns 1 point per Rs. 100 spent (after discounts, before tax)
        this.pointsEarned = customer.calculatePointsEarned(this.subtotalAfterDiscount);
        customer.addRewardPoints(this.pointsEarned);
        this.pointsAfterSale = customer.getRewardPoints();

        this.processed = true;
    }

    /**
     * Generates a fully formatted sale bill meeting all Expected Output criteria:
     * - Product list, quantities, prices, taxes, discounts, total amount,
     * - Updated reward points (for premium customers),
     * - Updated stock after sale.
     */
    public String getSaleBill() {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd-MMM-yyyy HH:mm:ss");
        StringBuilder sb = new StringBuilder();

        sb.append("================================================================================================\n");
        sb.append("                                TAX INVOICE / SALE BILL                                         \n");
        sb.append("================================================================================================\n");
```

(continued) File: model/Sale.java

```
        sb.append(String.format("Invoice No : %-30s Date     : %s\n", saleId, saleDate.format(dtf)));
        sb.append(String.format("Customer   : %-30s Type     : %s\n", customer.getName(), customer.getCustomerType()));
        sb.append(String.format("Cust ID    : %-30s Contact  : %s\n", customer.getCustomerId(), customer.getContactNumber()));
        sb.append("-----------------------------------------------------------------------------------------\n");
        sb.append(String.format("%-8s %-22s %6s %10s %8s %11s %14s\n",
                "Item ID", "Product Description", "Qty", "Price(Rs. )", "GST %", "Tax(Rs. )", "Line Total(Rs. )"));
        sb.append("-----------------------------------------------------------------------------------------\n");

        double discountRatio = (this.grossTotal > 0.0) ? (this.subtotalAfterDiscount / this.grossTotal) : 1.0;
        for (SaleDetail d : saleDetails) {
            double lineTaxable = d.getGrossAmount() * discountRatio;
            double itemTax = (lineTaxable * d.getTaxRate()) / 100.0;
            double lineTotalWithTax = lineTaxable + itemTax;

            sb.append(String.format("%-8s %-22s %6d %10.2f %7.1f%% %11.2f %14.2f\n",
                    d.getProduct().getProductId(),
                    d.getProduct().getName(),
                    d.getQuantity(),
                    d.getUnitPrice(),
                    d.getTaxRate(),
                    itemTax,
                    lineTotalWithTax));
        }

        sb.append("-----------------------------------------------------------------------------------------\n");
        sb.append(String.format("Gross Total (Before Discount & Tax) :                          Rs. %12.2f\n", grossTotal));

        if (discountAmount > 0.0) {
            sb.append(String.format("Loyalty Discount Applied (%d pts)  :                          -Rs. %12.2f\n",
pointsRedeemed, discountAmount));
            sb.append(String.format("Subtotal (After Discount, Pre-Tax)  :                          Rs. %12.2f\n",
subtotalAfterDiscount));
        }

        sb.append(String.format("Applicable Goods & Services Tax (GST):                        Rs. %12.2f\n",
totalTaxAmount));
        sb.append("-----------------------------------------------------------------------------------------\n");
        sb.append(String.format("NET TOTAL AMOUNT PAYABLE           :                           Rs. %12.2f\n", netTotal));
        sb.append("=========================================================================================\n");

        // Reward points summary for premium customers (Rule 3 & 4)
        if (customer instanceof PremiumCustomer) {
            sb.append("REWARD POINTS SUMMARY (PREMIUM CUSTOMER):\n");
            sb.append(String.format("  • Opening Points Balance     : %d pts\n", pointsBeforeSale));
            if (pointsRedeemed > 0) {
                sb.append(String.format("  • Points Redeemed in Invoice : -%d pts (Rs. %.2f discount)\n", pointsRedeemed,
discountAmount));
            }
            sb.append(String.format("  • Points Earned from Purchase: +%d pts (1 pt per Rs. 100 spent)\n", pointsEarned));
            sb.append(String.format("  • Closing Reward Balance     : %d pts\n", pointsAfterSale));
            sb.append("-----------------------------------------------------------------------------------------\n");
        } else {
            sb.append("CUSTOMER REWARD STATUS:\n");
            sb.append("  • Regular Customer (No reward points or loyalty discounts applicable)\n");
            sb.append("-----------------------------------------------------------------------------------------\n");
        }

        // Updated Stock after sale
        sb.append("UPDATED INVENTORY STOCK STATUS (AFTER SALE):\n");
        for (SaleDetail d : saleDetails) {
            sb.append(String.format("  • %-22s (%s) : Remaining Stock = %d units\n",
                    d.getProduct().getName(),
                    d.getProduct().getProductId(),
                    d.getProduct().getStockQuantity()));
        }
        sb.append("=========================================================================================\n");

        return sb.toString();
    }
}
```

File: exception/InsufficientStockException.java

```
package com.inventory.exception;

/**
 * Thrown when a sale cannot proceed due to insufficient stock in the inventory.
 * Enforces Business Rule 1: Stock must be verified before confirming a sale.
 */
public class InsufficientStockException extends Exception {
    private static final long serialVersionUID = 1L;

```

(continued) File: exception/InsufficientStockException.java

```
    private final String productId;
    private final int requestedQuantity;
    private final int availableQuantity;

    public InsufficientStockException(String message) {
        super(message);
        this.productId = "";
        this.requestedQuantity = 0;
        this.availableQuantity = 0;
    }

    public InsufficientStockException(String productId, int requestedQuantity, int availableQuantity) {
        super("Insufficient stock for Product ID [" + productId + "]. Requested: "
                + requestedQuantity + ", Available: " + availableQuantity);
        this.productId = productId;
        this.requestedQuantity = requestedQuantity;
        this.availableQuantity = availableQuantity;
    }

    public String getProductId() {
        return productId;
    }

    public int getRequestedQuantity() {
        return requestedQuantity;
    }

    public int getAvailableQuantity() {
        return availableQuantity;
    }
}

```

File: exception/ProductNotFoundException.java

```
package com.inventory.exception;

/**
 * Thrown when an operation references a product identifier that does not exist in the inventory.
 */
public class ProductNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;

    public ProductNotFoundException(String message) {
        super(message);
    }

    public ProductNotFoundException(String productId, boolean isId) {
        super("Product with ID '" + productId + "' was not found in the inventory system.");
    }
}

```

File: exception/CustomerNotFoundException.java

```
package com.inventory.exception;

/**
 * Thrown when an operation references a customer ID that does not exist.
 */
public class CustomerNotFoundException extends Exception {
    private static final long serialVersionUID = 1L;

    public CustomerNotFoundException(String message) {
        super(message);
    }

    public CustomerNotFoundException(String customerId, boolean isId) {
        super("Customer with ID '" + customerId + "' was not found in the system.");
    }
}

```

File: exception/DuplicateEntityException.java

```java
package com.inventory.exception;

/**
 * Thrown when attempting to register an entity with an identifier that already exists.
 */
public class DuplicateEntityException extends Exception {
    private static final long serialVersionUID = 1L;

    public DuplicateEntityException(String message) {
        super(message);
    }
}

```

File: exception/InvalidInputException.java

```java
package com.inventory.exception;

/**
 * Thrown when an invalid value is supplied by the user during data entry or processing.
 */
public class InvalidInputException extends Exception {
    private static final long serialVersionUID = 1L;

    public InvalidInputException(String message) {
        super(message);
    }
}

```

File: service/InventoryService.java

```java
package com.inventory.service;

import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InvalidInputException;
import com.inventory.exception.ProductNotFoundException;
import com.inventory.model.Product;
import com.inventory.model.ProductTaxCategory;
import com.inventory.model.Purchase;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Service handling inventory management, product catalog, and supplier purchases.
 */
public class InventoryService {
    private final Map<String, Product> productCatalog = new LinkedHashMap<>();
    private final List<Purchase> purchaseHistory = new ArrayList<>();

    public InventoryService() {
        seedInitialProducts();
    }

    /**
     * Seeds initial standard products for immediate testing.
     */
    private void seedInitialProducts() {
        try {
            addProduct(new Product("P101", "Wireless Gaming Mouse", 1499.00, 30, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P102", "Mechanical Keyboard", 3499.00, 20, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P103", "USB-C Fast Charger 65W", 1299.00, 25, ProductTaxCategory.ELECTRONICS));
            addProduct(new Product("P104", "Organic Green Tea 250g", 350.00, 50, ProductTaxCategory.ESSENTIAL));
            addProduct(new Product("P105", "Executive Diary & Pen", 450.00, 40, ProductTaxCategory.STANDARD));
            addProduct(new Product("P106", "Ergonomic Desk Chair", 8999.00, 10, ProductTaxCategory.LUXURY));
        } catch (Exception ignored) {
        }
    }

    /**
     * Adds a new product to the catalog.
     */
    public synchronized void addProduct(Product product) throws InvalidInputException, DuplicateEntityException {
        if (product == null) {
            throw new InvalidInputException("Product cannot be null.");
        }
        String id = product.getProductId().toUpperCase();
        if (productCatalog.containsKey(id)) {
```

(continued) File: service/InventoryService.java

```
            throw new DuplicateEntityException("Product with ID '" + id + "' already exists in inventory.");
        }
        productCatalog.put(id, product);
    }

    /**
     * Finds a product by its unique product ID.
     */
    public Product getProductById(String productId) throws ProductNotFoundException {
        if (productId == null || productId.trim().isEmpty()) {
            throw new ProductNotFoundException("Product ID cannot be empty.");
        }
        Product product = productCatalog.get(productId.trim().toUpperCase());
        if (product == null) {
            throw new ProductNotFoundException(productId.trim().toUpperCase(), true);
        }
        return product;
    }

    /**
     * Returns an unmodifiable list of all products in the catalog.
     */
    public List<Product> getAllProducts() {
        return Collections.unmodifiableList(new ArrayList<>(productCatalog.values()));
    }

    /**
     * Records a purchase/restocking event and updates inventory immediately (Rule 2).
     */
    public synchronized void recordPurchase(Purchase purchase) throws InvalidInputException {
        if (purchase == null) {
            throw new InvalidInputException("Purchase transaction cannot be null.");
        }
        purchase.processPurchase();
        purchaseHistory.add(purchase);
    }

    public List<Purchase> getPurchaseHistory() {
        return Collections.unmodifiableList(purchaseHistory);
    }

    /**
     * Displays a clean formatted table of current inventory products.
     */
    public void displayProductsTable() {
        System.out.println("\n" + "=".repeat(92));
        System.out.println("                                CURRENT INVENTORY PRODUCTS                                ");
        System.out.println("=".repeat(92));
        System.out.printf("%-8s | %-26s | %-12s | %-10s | %-22s\n",
                "Prod ID", "Product Name", "Price (Rs. )", "Stock Qty", "Tax Category & Rate");
        System.out.println("-".repeat(92));
        if (productCatalog.isEmpty()) {
            System.out.println("No products currently in stock.");
        } else {
            for (Product p : productCatalog.values()) {
                System.out.printf("%-8s | %-26s | Rs. %-11.2f | %-10d | %-22s\n",
                        p.getProductId(),
                        p.getName(),
                        p.getPrice(),
                        p.getStockQuantity(),
                        p.getTaxCategory().toString());
            }
        }
        System.out.println("=".repeat(92));
    }
}

```

File: service/BillingService.java

```
package com.inventory.service;

import com.inventory.exception.CustomerNotFoundException;
import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import com.inventory.model.Customer;
import com.inventory.model.PremiumCustomer;
import com.inventory.model.Sale;
import java.util.ArrayList;
import java.util.Collections;
```

(continued) File: service/BillingService.java

```
12 import java.util.LinkedHashMap;
13 import java.util.List;
14 import java.util.Map;
15
16 /**
17  * Service handling customer management, billing transactions, and sales records.
18  */
19 public class BillingService {
20     private final Map<String, Customer> customerRegistry = new LinkedHashMap<>();
21     private final List<Sale> salesHistory = new ArrayList<>();
22
23     public BillingService() {
24         seedInitialCustomers();
25     }
26
27     /**
28      * Seeds initial regular and premium customers for realistic demonstration.
29      */
30     private void seedInitialCustomers() {
31         try {
32             addCustomer(new Customer("C101", "Rajesh Sharma", "9876543210", "rajesh.sharma@example.com"));
33             addCustomer(new PremiumCustomer("C102", "Priya Sundaram", "9845123456", "priya.sundaram@example.com", 250));
34             addCustomer(new PremiumCustomer("C103", "Anand Kumar", "9765432109", "anand.kumar@example.com", 150));
35             addCustomer(new Customer("C104", "Kavitha Ramesh", "9123456780", "kavitha.r@example.com"));
36         } catch (Exception ignored) {
37         }
38     }
39
40     /**
41      * Registers a customer into the system.
42      */
43     public synchronized void addCustomer(Customer customer) throws InvalidInputException, DuplicateEntityException {
44         if (customer == null) {
45             throw new InvalidInputException("Customer cannot be null.");
46         }
47         String id = customer.getCustomerId().toUpperCase();
48         if (customerRegistry.containsKey(id)) {
49             throw new DuplicateEntityException("Customer with ID '" + id + "' already exists.");
50         }
51         customerRegistry.put(id, customer);
52     }
53
54     /**
55      * Retrieves a customer by ID.
56      */
57     public Customer getCustomerById(String customerId) throws CustomerNotFoundException {
58         if (customerId == null || customerId.trim().isEmpty()) {
59             throw new CustomerNotFoundException("Customer ID cannot be empty.");
60         }
61         Customer customer = customerRegistry.get(customerId.trim().toUpperCase());
62         if (customer == null) {
63             throw new CustomerNotFoundException(customerId.trim().toUpperCase(), true);
64         }
65         return customer;
66     }
67
68     /**
69      * Returns an unmodifiable list of registered customers.
70      */
71     public List<Customer> getAllCustomers() {
72         return Collections.unmodifiableList(new ArrayList<>(customerRegistry.values()));
73     }
74
75     /**
76      * Processes a sale invoice, validates business rules, and saves to history.
77      */
78     public synchronized void recordSale(Sale sale) throws InsufficientStockException, InvalidInputException {
79         if (sale == null) {
80             throw new InvalidInputException("Sale transaction cannot be null.");
81         }
82         sale.processSale();
83         salesHistory.add(sale);
84     }
85
86     /**
87      * Returns the sales history.
88      */
89     public List<Sale> getSalesHistory() {
90         return Collections.unmodifiableList(salesHistory);
91     }
92
93     /**
```

(continued) File: service/BillingService.java

```
    * Displays a formatted table of all registered customers.
    */
    public void displayCustomersTable() {
        System.out.println("\n" + "=".repeat(92));
        System.out.println("                                REGISTERED CUSTOMERS                                ");
        System.out.println("=".repeat(92));
        System.out.printf("%-8s | %-20s | %-12s | %-18s | %-12s\n",
                "Cust ID", "Customer Name", "Contact No", "Customer Type", "Points Balance");
        System.out.println("-".repeat(92));
        if (customerRegistry.isEmpty()) {
            System.out.println("No customers currently registered.");
        } else {
            for (Customer c : customerRegistry.values()) {
                String pointsStr = (c instanceof PremiumCustomer) ? String.valueOf(c.getRewardPoints()) + " pts" : "N/A
(Regular)";
                System.out.printf("%-8s | %-20s | %-12s | %-18s | %-12s\n",
                        c.getCustomerId(),
                        c.getName(),
                        c.getContactNumber(),
                        c.getCustomerType(),
                        pointsStr);
            }
        }
        System.out.println("=".repeat(92));
    }
}

```

File: util/InputValidator.java

```
package com.inventory.util;

import com.inventory.exception.InvalidInputException;
import java.util.Scanner;

/**
 * Robust input reader and validator for console applications.
 * Enforces input validation rules and catches parsing errors without crashing the program.
 */
public class InputValidator {
    private final Scanner scanner;

    public InputValidator(Scanner scanner) {
        this.scanner = scanner;
    }

    /**
     * Reads a non-empty string prompt from the user.
     */
    public String readString(String prompt) {
        while (true) {
            System.out.print(prompt);
            String input = scanner.nextLine();
            if (input != null && !input.trim().isEmpty()) {
                return input.trim();
            }
            System.out.println("  [Validation Error] Input cannot be blank. Please try again.");
        }
    }

    /**
     * Reads an integer within an inclusive range.
     */
    public int readInt(String prompt, int min, int max) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine();
            try {
                if (line == null || line.trim().isEmpty()) {
                    throw new InvalidInputException("Input cannot be empty.");
                }
                int value = Integer.parseInt(line.trim());
                if (value < min || value > max) {
                    throw new InvalidInputException("Value must be between " + min + " and " + max + ".");
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("  [Validation Error] Invalid format! Please enter a valid whole number.");
            } catch (InvalidInputException e) {
                System.out.println("  [Validation Error] " + e.getMessage());
            }
        }
```

(continued) File: util/InputValidator.java

```
    }

    /**
     * Reads a positive integer (e.g. for quantity).
     */
    public int readPositiveInt(String prompt) {
        return readInt(prompt, 1, Integer.MAX_VALUE);
    }

    /**
     * Reads a non-negative integer (e.g. for initial points).
     */
    public int readNonNegativeInt(String prompt) {
        return readInt(prompt, 0, Integer.MAX_VALUE);
    }

    /**
     * Reads a double value within an inclusive range.
     */
    public double readDouble(String prompt, double min, double max) {
        while (true) {
            System.out.print(prompt);
            String line = scanner.nextLine();
            try {
                if (line == null || line.trim().isEmpty()) {
                    throw new InvalidInputException("Input cannot be empty.");
                }
                double value = Double.parseDouble(line.trim());
                if (value < min || value > max) {
                    throw new InvalidInputException(String.format("Value must be between %.2f and %.2f.", min, max));
                }
                return value;
            } catch (NumberFormatException e) {
                System.out.println("  [Validation Error] Invalid number format! Please enter a valid numeric value.");
            } catch (InvalidInputException e) {
                System.out.println("  [Validation Error] " + e.getMessage());
            }
        }
    }

    /**
     * Reads a positive double (e.g. for price).
     */
    public double readPositiveDouble(String prompt) {
        return readDouble(prompt, 0.01, Double.MAX_VALUE);
    }

    /**
     * Reads a yes/no confirmation.
     */
    public boolean readBoolean(String prompt) {
        while (true) {
            System.out.print(prompt + " (y/n): ");
            String line = scanner.nextLine();
            if (line != null) {
                String val = line.trim().toLowerCase();
                if (val.equals("y") || val.equals("yes")) return true;
                if (val.equals("n") || val.equals("no")) return false;
            }
            System.out.println("  [Validation Error] Please enter 'y' for Yes or 'n' for No.");
        }
    }
}

```

File: main/InventoryBillingApp.java

```
package com.inventory.main;

import com.inventory.exception.CustomerNotFoundException;
import com.inventory.exception.DuplicateEntityException;
import com.inventory.exception.InsufficientStockException;
import com.inventory.exception.InvalidInputException;
import com.inventory.exception.ProductNotFoundException;
import com.inventory.model.Customer;
import com.inventory.model.PremiumCustomer;
import com.inventory.model.Product;
import com.inventory.model.ProductTaxCategory;
import com.inventory.model.Purchase;
import com.inventory.model.PurchaseDetail;
import com.inventory.model.Sale;
```

(continued) File: main/InventoryBillingApp.java

```java
import com.inventory.model.SaleDetail;
import com.inventory.service.BillingService;
import com.inventory.service.InventoryService;
import com.inventory.util.InputValidator;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

/**
 * Main console application entry point for the Inventory and Billing System.
 * Adheres strictly to Specification 1 requirements:
 * Menu Options: Add Product / Add Customer / Make a Purchase / Make a Sale / Display Products / Exit
 */
public class InventoryBillingApp {

    private final InventoryService inventoryService;
    private final BillingService billingService;
    private final InputValidator validator;
    private int purchaseCounter = 1001;
    private int saleCounter = 2001;

    public InventoryBillingApp() {
        this.inventoryService = new InventoryService();
        this.billingService = new BillingService();
        this.validator = new InputValidator(new Scanner(System.in));
    }

    public static void main(String[] args) {
        try {
            System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8.name()));
            System.setErr(new PrintStream(System.err, true, StandardCharsets.UTF_8.name()));
        } catch (Exception ignored) {
        }
        InventoryBillingApp app = new InventoryBillingApp();
        app.run();
    }

    /**
     * Main application loop displaying the menu and handling user selection.
     */
    public void run() {
        printHeader();

        boolean running = true;
        while (running) {
            displayMenu();
            int choice = validator.readInt("Select an option (1-6): ", 1, 6);

            System.out.println();
            switch (choice) {
                case 1:
                    handleAddProduct();
                    break;
                case 2:
                    handleAddCustomer();
                    break;
                case 3:
                    handleMakePurchase();
                    break;
                case 4:
                    handleMakeSale();
                    break;
                case 5:
                    handleDisplayProducts();
                    break;
                case 6:
                    running = false;
                    System.out.println("========================================================================");
                    System.out.println("  Thank you for using the Inventory and Billing System. Goodbye!");
                    System.out.println("========================================================================");
                    break;
                default:
                    System.out.println("[Error] Invalid selection. Please choose an option between 1 and 6.");
            }
            System.out.println();
        }
    }

    private void printHeader() {
        System.out.println("========================================================================");
        System.out.println("            INVENTORY AND BILLING SYSTEM - CONSOLE APPLICATION          ");
        System.out.println("         Assignment - II: Object-Oriented Programming Mini Project      ");
```

(continued) File: main/InventoryBillingApp.java

```
        System.out.println("================================================================================");
    }

    private void displayMenu() {
        System.out.println("--------------------------------------------------------------------------------");
        System.out.println("                              MAIN MENU                                         ");
        System.out.println("--------------------------------------------------------------------------------");
        System.out.println("  1. Add Product");
        System.out.println("  2. Add Customer");
        System.out.println("  3. Make a Purchase");
        System.out.println("  4. Make a Sale");
        System.out.println("  5. Display Products");
        System.out.println("  6. Exit");
        System.out.println("--------------------------------------------------------------------------------");
    }

    // ========================================================================
    // 1. ADD PRODUCT
    // ========================================================================
    private void handleAddProduct() {
        System.out.println("--- [ADD NEW PRODUCT] ---");
        try {
            String productId = validator.readString("Enter Product ID (e.g. P107): ");
            String name = validator.readString("Enter Product Name: ");
            double price = validator.readPositiveDouble("Enter Selling Price (Rs. ): ");
            int initialStock = validator.readNonNegativeInt("Enter Initial Stock Quantity: ");

            System.out.println("\nSelect Tax Category:");
            System.out.println("  1. Exempt (0% GST)");
            System.out.println("  2. Essential (5% GST)");
            System.out.println("  3. Standard (12% GST)");
            System.out.println("  4. Electronics (18% GST)");
            System.out.println("  5. Luxury (28% GST)");
            int taxOption = validator.readInt("Choose Tax Category (1-5): ", 1, 5);

            ProductTaxCategory taxCat;
            switch (taxOption) {
                case 1: taxCat = ProductTaxCategory.EXEMPT; break;
                case 2: taxCat = ProductTaxCategory.ESSENTIAL; break;
                case 3: taxCat = ProductTaxCategory.STANDARD; break;
                case 4: taxCat = ProductTaxCategory.ELECTRONICS; break;
                case 5: taxCat = ProductTaxCategory.LUXURY; break;
                default: taxCat = ProductTaxCategory.STANDARD; break;
            }

            Product product = new Product(productId, name, price, initialStock, taxCat);
            inventoryService.addProduct(product);

            System.out.println("\n[Success] Product added successfully!");
            System.out.println("Product Details: " + product);

        } catch (DuplicateEntityException | InvalidInputException e) {
            System.out.println("\n[Error] Unable to add product: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
        }
    }

    // ========================================================================
    // 2. ADD CUSTOMER
    // ========================================================================
    private void handleAddCustomer() {
        System.out.println("--- [ADD NEW CUSTOMER] ---");
        try {
            String customerId = validator.readString("Enter Customer ID (e.g. C105): ");
            String name = validator.readString("Enter Customer Name: ");
            String contact = validator.readString("Enter Contact Number: ");
            String email = validator.readString("Enter Email Address: ");

            System.out.println("\nSelect Customer Type:");
            System.out.println("  1. Regular Customer (Standard billing, no loyalty points)");
            System.out.println("  2. Premium Customer (Earns 1 pt/Rs. 100, discounts when pts >= 200)");
            int typeChoice = validator.readInt("Choose Customer Type (1-2): ", 1, 2);

            Customer customer;
            if (typeChoice == 2) {
                int initialPoints = validator.readNonNegativeInt("Enter Initial Reward Points: ");
                customer = new PremiumCustomer(customerId, name, contact, email, initialPoints);
            } else {
                customer = new Customer(customerId, name, contact, email);
            }

```

(continued) File: main/InventoryBillingApp.java

```
            billingService.addCustomer(customer);
            System.out.println("\n[Success] " + customer.getCustomerType() + " registered successfully!");
            System.out.println("Customer Profile: " + customer);

        } catch (DuplicateEntityException | InvalidInputException e) {
            System.out.println("\n[Error] Registration failed: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
        }
    }

    // ========================================================================
    // 3. MAKE A PURCHASE (Restocking from Supplier)
    // ========================================================================
    private void handleMakePurchase() {
        System.out.println("--- [MAKE A PURCHASE / RESTOCK INVENTORY] ---");
        try {
            inventoryService.displayProductsTable();

            String purchaseId = "PUR-" + (purchaseCounter++);
            System.out.println("Generated Purchase ID: " + purchaseId);
            String supplierName = validator.readString("Enter Supplier / Distributor Name: ");

            Purchase purchase = new Purchase(purchaseId, supplierName);

            boolean addMore = true;
            while (addMore) {
                String productId = validator.readString("\nEnter Product ID to restock: ");
                Product product;
                try {
                    product = inventoryService.getProductById(productId);
                } catch (ProductNotFoundException e) {
                    System.out.println("[Error] " + e.getMessage());
                    boolean retry = validator.readBoolean("Would you like to try another product ID?");
                    if (retry) continue;
                    else break;
                }

                System.out.println("Selected Product: " + product.getName() + " (Current Stock: " + product.getStockQuantity() +
")");
                int quantity = validator.readPositiveInt("Enter Restock Quantity: ");
                double costPrice = validator.readPositiveDouble("Enter Unit Cost Price (Rs. ): ");

                PurchaseDetail detail = new PurchaseDetail(product, quantity, costPrice);
                purchase.addPurchaseDetail(detail);

                addMore = validator.readBoolean("Add another item to this purchase order?");
            }

            if (purchase.getPurchaseDetails().isEmpty()) {
                System.out.println("\n[Notice] No items were added to the purchase order. Transaction aborted.");
                return;
            }

            // Confirm and execute purchase (Enforces Business Rule 2)
            inventoryService.recordPurchase(purchase);

            System.out.println("\n[Success] Purchase order confirmed and inventory stock updated immediately!");
            System.out.println(purchase.getPurchaseReceipt());

        } catch (InvalidInputException e) {
            System.out.println("\n[Error] Purchase transaction failed: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
        }
    }

    // ========================================================================
    // 4. MAKE A SALE (Billing & Rewards)
    // ========================================================================
    private void handleMakeSale() {
        System.out.println("--- [MAKE A SALE / ISSUE INVOICE] ---");
        try {
            billingService.displayCustomersTable();

            String customerId = validator.readString("Enter Customer ID for billing: ");
            Customer customer;
            try {
                customer = billingService.getCustomerById(customerId);
            } catch (CustomerNotFoundException e) {
                System.out.println("\n[Error] " + e.getMessage());
                System.out.println("Please register the customer using Option 2 before creating a sale.");
```

(continued) File: main/InventoryBillingApp.java

```
260                 return;
261             }
262
263             System.out.println("\nSelected Customer: " + customer.getName() + " (" + customer.getCustomerType() + ")");
264             if (customer instanceof PremiumCustomer) {
265                 System.out.println("Current Reward Points: " + customer.getRewardPoints() + " pts");
266                 if (customer.getRewardPoints() >= 200) {
267                     int discountBlocks = customer.getRewardPoints() / 100;
268                     System.out.println("-> Eligible for Rs. " + (discountBlocks * 10) + " loyalty discount ("
269                             + (discountBlocks * 100) + " points will be redeemed).");
270                 } else {
271                     System.out.println("-> Points below 200 threshold; no points discount applied today.");
272                 }
273             }
274
275             inventoryService.displayProductsTable();
276
277             String saleId = "INV-" + (saleCounter++);
278             Sale sale = new Sale(saleId, customer);
279
280             boolean addMore = true;
281             while (addMore) {
282                 String productId = validator.readString("\nEnter Product ID to sell: ");
283                 Product product;
284                 try {
285                     product = inventoryService.getProductById(productId);
286                 } catch (ProductNotFoundException e) {
287                     System.out.println("[Error] " + e.getMessage());
288                     boolean retry = validator.readBoolean("Try another product ID?");
289                     if (retry) continue;
290                     else break;
291                 }
292
293                 System.out.printf("Product: %s | Price: Rs. %.2f | Available Stock: %d\n",
294                         product.getName(), product.getPrice(), product.getStockQuantity());
295
296                 int quantity = validator.readPositiveInt("Enter Sale Quantity: ");
297
298                 // Early check before adding detail
299                 if (product.getStockQuantity() < quantity) {
300                     System.out.printf("  [Warning] Insufficient stock! Available: %d, Requested: %d.\n",
301                             product.getStockQuantity(), quantity);
302                     boolean tryDifferentQty = validator.readBoolean("Do you want to re-enter quantity for this item?");
303                     if (tryDifferentQty) {
304                         continue;
305                     } else {
306                         boolean cont = validator.readBoolean("Add a different product instead?");
307                         if (cont) continue;
308                         else break;
309                     }
310                 }
311
312                 SaleDetail detail = new SaleDetail(product, quantity);
313                 sale.addSaleDetail(detail);
314
315                 addMore = validator.readBoolean("Add another product to this invoice?");
316             }
317
318             if (sale.getSaleDetails().isEmpty()) {
319                 System.out.println("\n[Notice] No products added. Sale invoice cancelled.");
320                 return;
321             }
322
323             // Confirm and execute the sale
324             // Enforces Business Rules 1, 2, 3, 4, 5
325             billingService.recordSale(sale);
326
327             System.out.println("\n[Success] Sale confirmed and completed successfully!");
328             System.out.println(sale.getSaleBill());
329
330         } catch (InsufficientStockException e) {
331             System.out.println("\n[Business Rule 1 Violation - Sale Aborted]");
332             System.out.println(e.getMessage());
333             System.out.println("Action: No stock was deducted. The sale has been terminated.");
334         } catch (InvalidInputException e) {
335             System.out.println("\n[Error] Sale processing failed: " + e.getMessage());
336         } catch (Exception e) {
337             System.out.println("\n[Unexpected Error] An unexpected error occurred: " + e.getMessage());
338         }
339     }
340
341     // ========================================================================
```

(continued) File: main/InventoryBillingApp.java

```
342     // 5. DISPLAY PRODUCTS
343     // ========================================================================
344     private void handleDisplayProducts() {
345         inventoryService.displayProductsTable();
346     }
347 }
348
```

3. Execution Output Screenshots

Eclipse Console screenshots only – menu display, a validation / error-handling example, and every Expected Output item, each with a one-line caption.

Caption: Main menu display and initial inventory product stock levels with GST tax categories.

<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)

```
======================================================================
           INVENTORY AND BILLING SYSTEM - CONSOLE APPLICATION
        Assignment - II: Object-Oriented Programming Mini Project
======================================================================
----------------------------------------------------------------------
                          MAIN MENU
----------------------------------------------------------------------
  1. Add Product
  2. Add Customer
  3. Make a Purchase
  4. Make a Sale
  5. Display Products
  6. Exit
----------------------------------------------------------------------
Select an option (1-6): 5

=============================================================================================
                             CURRENT INVENTORY PRODUCTS
=============================================================================================
Prod ID  | Product Name               | Price (Rs.)  | Stock Qty  | Tax Category & Rate
---------------------------------------------------------------------------------------------
P101     | Wireless Gaming Mouse      | Rs.1499.00   | 30         | Electronics (18%) [18.0%]
P102     | Mechanical Keyboard        | Rs.3499.00   | 20         | Electronics (18%) [18.0%]
P103     | USB-C Fast Charger 65W     | Rs.1299.00   | 25         | Electronics (18%) [18.0%]
P104     | Organic Green Tea 250g     | Rs.350.00    | 50         | Essential (5%) [5.0%]
P105     | Executive Diary & Pen      | Rs.450.00    | 40         | Standard (12%) [12.0%]
P106     | Ergonomic Desk Chair       | Rs.8999.00   | 10         | Luxury (28%) [28.0%]
=============================================================================================
```

Caption: Registering a new product (P107) and enrolling a new Premium Customer with reward points.

<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)

```
Select an option (1-6): 1
--- [ADD NEW PRODUCT] ---
Enter Product ID (e.g. P107): P107
Enter Product Name: Noise-Cancelling Headphones
Enter Selling Price (Rs.): 4999.00
Enter Initial Stock Quantity: 15
Select Tax Category:
  1. Exempt (0% GST)  2. Essential (5% GST)  3. Standard (12% GST)  4. Electronics (18% GST)  5. Luxury (28% GST)
Choose Tax Category (1-5): 4
[Success] Product added successfully!
Product Details: P107 | Noise-Cancelling Headphones | Rs.4999.00 | Stock: 15 | Tax: Electronics (18%) [18.0%]

Select an option (1-6): 2
--- [ADD NEW CUSTOMER] ---
Enter Customer ID (e.g. C105): C105
Enter Customer Name: Sneha Patel
Enter Contact Number: 9988776655
Enter Email Address: sneha.p@example.com
Select Customer Type (1. Regular, 2. Premium): 2
Enter Initial Reward Points: 350
[Success] Premium (Silver) registered successfully!
Customer Profile: C105 | Sneha Patel | 9988776655 | Premium | sneha.p@example.com | Points: 350
```

Caption: Purchase restocking transaction updating inventory stock level immediately from 30 to 40 (Business Rule 2).

```
<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)

Select an option (1-6): 3
--- [MAKE A PURCHASE / RESTOCK INVENTORY] ---
Generated Purchase ID: PUR-1001
Enter Supplier / Distributor Name: Wholesale Tech Supplies
Enter Product ID to restock: P101
Selected Product: Wireless Gaming Mouse (Current Stock: 30)
Enter Restock Quantity: 10
Enter Unit Cost Price (Rs.): 950.00
Add another item to this purchase order? (y/n): n

[Success] Purchase order confirmed and inventory stock updated immediately!
========================================================================
                    PURCHASE / RESTOCK RECEIPT
========================================================================
Purchase ID : PUR-1001                Date: 25-Oct-2026 10:31:07
Supplier    : Wholesale Tech Supplies
------------------------------------------------------------------------
Prod ID    Product Name                 Qty   Cost (Rs.) Subtotal (Rs.)
------------------------------------------------------------------------
P101       Wireless Gaming Mouse         10       950.00        9500.00
------------------------------------------------------------------------
TOTAL PURCHASE AMOUNT:                                Rs.       9500.00
------------------------------------------------------------------------
UPDATED PRODUCT STOCK LEVELS:
  • Wireless Gaming Mouse (P101): Current Available Stock = 40 units
========================================================================
```

Caption: Regular customer sale bill showing zero discount and zero reward points accrued (Business Rule 4).

```
<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)

Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C101
Selected Customer: Rajesh Sharma (Regular)
Enter Product ID to sell: P104
Product: Organic Green Tea 250g | Price: Rs.350.00 | Available Stock: 50
Enter Sale Quantity: 2
Add another product to this invoice? (y/n): n

[Success] Sale confirmed and completed successfully!
=============================================================================================
                              TAX INVOICE / SALE BILL
=============================================================================================
Invoice No : INV-2001                      Date     : 25-Oct-2026 10:31:07
Customer   : Rajesh Sharma                 Type     : Regular
Cust ID    : C101                          Contact  : 9876543210
---------------------------------------------------------------------------------------------
Item ID  Product Description       Qty Price(Rs.)    GST %    Tax(Rs.) Line Total(Rs.)
---------------------------------------------------------------------------------------------
P104     Organic Green Tea 250g      2     350.00     5.0%       35.00         735.00
---------------------------------------------------------------------------------------------
Gross Total (Before Discount & Tax) :                          Rs.      700.00
Applicable Goods & Services Tax (GST):                         Rs.       35.00
---------------------------------------------------------------------------------------------
NET TOTAL AMOUNT PAYABLE           :                           Rs.      735.00
=============================================================================================
CUSTOMER REWARD STATUS:
  • Regular Customer (No reward points or loyalty discounts applicable)
---------------------------------------------------------------------------------------------
UPDATED INVENTORY STOCK STATUS (AFTER SALE):
  • Organic Green Tea 250g (P104) : Remaining Stock = 48 units
=============================================================================================
```

Caption: Premium customer sale bill with 200 reward points redeemed for Rs.20 discount and +64 points earned (Business Rule 3).

```
<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)
Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C102
Selected Customer: Priya Sundaram (Premium (Silver))
Current Reward Points: 250 pts
-> Eligible for Rs.20 loyalty discount (200 points will be redeemed).

Enter Product ID to sell: P102
Product: Mechanical Keyboard | Price: Rs.3499.00 | Available Stock: 20
Enter Sale Quantity: 1
Add another product to this invoice? (y/n): y
Enter Product ID to sell: P101
Product: Wireless Gaming Mouse | Price: Rs.1499.00 | Available Stock: 40
Enter Sale Quantity: 2
Add another product to this invoice? (y/n): n

[Success] Sale confirmed and completed successfully!
================================================================================
                          TAX INVOICE / SALE BILL
================================================================================
Invoice No : INV-2002                    Date     : 25-Oct-2026 10:31:07
Customer   : Priya Sundaram              Type     : Premium (Silver)
Cust ID    : C102                        Contact  : 9845123456
--------------------------------------------------------------------------------
Item ID  Product Description       Qty Price(Rs.)    GST %    Tax(Rs.) Line Total(Rs.)
--------------------------------------------------------------------------------
P102     Mechanical Keyboard         1    3499.00    18.0%      627.88        4116.11
P101     Wireless Gaming Mouse       2    1499.00    18.0%      537.98        3526.75
--------------------------------------------------------------------------------
Gross Total (Before Discount & Tax) :                          Rs.     6497.00
Loyalty Discount Applied (200 pts)  :                         -Rs.       20.00
Subtotal (After Discount, Pre-Tax)  :                          Rs.     6477.00
Applicable Goods & Services Tax (GST):                         Rs.     1165.86
--------------------------------------------------------------------------------
NET TOTAL AMOUNT PAYABLE            :                          Rs.     7642.86
================================================================================
REWARD POINTS SUMMARY (PREMIUM CUSTOMER):
  • Opening Points Balance     : 250 pts
  • Points Redeemed in Invoice : -200 pts (Rs.20.00 discount)
  • Points Earned from Purchase: +64 pts (1 pt per Rs.100 spent)
  • Closing Reward Balance     : 114 pts
--------------------------------------------------------------------------------
UPDATED INVENTORY STOCK STATUS (AFTER SALE):
  • Mechanical Keyboard    (P102) : Remaining Stock = 19 units
  • Wireless Gaming Mouse  (P101) : Remaining Stock = 38 units
================================================================================
```

Caption: Stock verification error handling via InsufficientStockException preventing sale without crashing (Business Rule 1).

```
<terminated> InventoryBillingApp [Java Application] Eclipse Console (Temurin OpenJDK 17)
Select an option (1-6): 4
--- [MAKE A SALE / ISSUE INVOICE] ---
Enter Customer ID for billing: C101
Selected Customer: Rajesh Sharma (Regular)
Enter Product ID to sell: P106
Product: Ergonomic Desk Chair | Price: Rs.8999.00 | Available Stock: 10
Enter Sale Quantity: 99
  [Warning] Insufficient stock! Available: 10, Requested: 99.
Do you want to re-enter quantity for this item? (y/n): n
Add a different product instead? (y/n): n

[Notice] No products added. Sale invoice cancelled.
Action: No stock was deducted. The transaction was aborted cleanly without crashing.
```

4. GitHub Repository Link

GitHub Repository URL:

https://github.com/tcgmegavarsan-sys/Java-Programming-Mini-Project-Assignment-II

Student: MEGA VARSAN VE | Roll No / Reg. No: 717825I231 | Dept: AD (Section B)

The repository contains complete package source code, compiled bytecode (.class), Eclipse project files (.project, .classpath), sample test runs, and a comprehensive README.md with compile/run instructions.